

Hello everyone and miss Sang, my name is Duong Thi Quynh Nhu and I am here to talk about our project, Sentimental analysis chatbot. Our project consists of 3 members, me, Danny Donnelly and Justin Butler from Udayton University. Sentimental analysis has become a popular research field in recent years, applying the power of natural language processing of Artificial Intelligence to determine emotions conveyed in the text to enhance marketing strategy evaluations of businesses. Therefore, our group also wants to contribute to this research field with our project. The dataset we are using is the Emotion dataset from Kaggle with over 417000 different data instances which are categorized into 6 different classes (sadness, joy, love, anger, fear and surprise). The dataset is obtained mainly from Twitter status.

Since our dataset is mainly strings, the pre-processing phases include 6 main steps. Firstly, we eliminate extra spaces and expand contractions. Because our dataset is mainly status online, it contains people 's usernames as well as stop words which will increase the unnecessary noise for our dataset, removing user handles and stopwords are our next step. The final step is lemmatizing and checking for miss spelled words in the dataset.

As shown in the picture above, although our dataset is massive, it is extremely unbalanced with the majority being classes 0 and 1 and the minority being class 2 and 5. This behaviour of the dataset has strongly impacted our model in a negative way. The next line graph illustrates the average length of meaningful tokens over the whole dataset. As I show over here, the meaningful token amount of each data instance is relatively small with the average being 5 to 6 meaningful tokens per sentence. However, the maximum of meaningful tokens is 62 while the minimum is 1. Moreover, we would like to analyze the specific meaningful tokens of each label. As you can see over here, label fear (4), anger (3) and sadness (0) have the most distinguishing specific tokens, that explain why the accuracy specific for those labels are higher compared to the others. Label joy (1) has nearly similar specific tokens with label love (2) such as accepted, lovely , etc. The most concerning class is class 5 (surprise) since their distinguished points are not too different compared to other classes with some confused words such as body, face, etc.

The algorithm we explore for this dataset is Logistic regression due to its simplicity and fast pace in training and testing. Since our problem is a multiclass, we utilize softmax loss function and for the optimizing method, we apply stochastic gradient descent (which is the updated version compared with the ordinal gradient suitable for big datasets (like our Emotion dataset). The main difference between stochastic gradient descent and gradient descent is that it only use a portion of the dataset to update weight value

Before applying the algorithm, our first step is vectorizing the tokens and the method we choose is Term Frequency - Inverse Document Frequency Vectorization with two main formulas shown on the board, the first one illustrates the importance of word in a single document and the second one illustrates the importance of the word across all documents and penalizes common words. On the default, we extract every single word and use the Euclidean formula to normalize the document vector to have norm of 1. After applying the algorithm by sklearn library from Python, the results are positive with 88.17% for accuracy, 88.3% for precision and 88.17% for recall overall. However, as I mentioned before, the unbalanced attribute of the dataset as well as the

indistinguishable of specific key points, the specific statistics for each class are not equivalent. As shown below, the most confident class that our model can give is class 0, 1 and 3 with very high recall as well as precision. However, due to the similarity of specific data points of class 1 and 2, a high amount of class 2 are usually miss labeled with class 1 (but not in the reverse side because class 1 has far more instances compared to class 2) which leads to the low of recall in class 2 and precision in class 1. Class 5 also possesses a low point on recall due to its lack of instances compared with other labels.

Therefore, we come to the strategy to improve the performance of logistic regression on this dataset. Firstly, we have some modification on the vectorization by applying the sublinear scaling to reduce the impact of high-frequency words and improve the sensitivity towards words that appear less frequently by using logarithm function, as shown in the formula here. Now, not only extract words in unigram, we extract them in bigrams which means we keep two words if they support each other. Finally, the top 50000 features are kept based on their TF-IDF score to reduce dimensionality and improve efficiency.

Moreover, we also tune the hyper parameters of the logistic regression such as regularization value, penalty and loss function by applying grid search cv library. After tuning, the result illustrates that the best hyper parameter for this dataset is 0.0001 for regularization, loss function is modified huber and penalty used is l2 (Euclidean). Lastly, we add 'balanced' for class_weight hyper parameter to increase the weight of small class (in this case, class 5) to penalize more when miss-classified small class.

After modification, the statistics have significantly improved with 90.85% for accuracy, 91.74% for precision and 91.06% for recall which illustrates that we are on the right track for our project. As you can see over here, the performance for all classes have improved significantly. The percentage of class 2 being miss classified to class 1 has reduced dramatically. In overall, the recall statistics of minority class have improved in exchange with slight precision decrease.

As you can see over here are the Precision-recall curve and roc curve. As shown in the two graphs, the precision curve and ROC curve illustrate that label 0, 1 and 3 precision curves are extraordinarily smooth with nearly approach 1 indicating that our model is extremely confident in defining these 3 classes. However, in class 2 and 5, the precision curves are fluctuating and reduce when recall value increases shows that the model is imbalanced for those 2 classes due to the shortage of instances. the roc curve shows that our models works well on all five classes however, the result of roc curve is affected by the unbalanced of the dataset so in this case, the precision recall curve is more reliable.

Beside high performance, one competitive edge of logistic regression is its short time complexity in training and testing. The time complexity of logistic regression, especially with stochastic descent regression is always an advantage when compared with other more complicated algorithms with Training time complexity equals $O(e \times n \times d)$ with e is number of epochs, n is training samples and d is number of features and Testing time complexity is $O(n \times d)$ with n is number of test samples and d is number of features.

Lastly, I also compared our method with other methods existing on the field. We compared naive bayes and Light Gradient Boosting Machine (a modification from original gbm by microsoft), As you can see over here, both complementN naive bayes and multinomial naive bayes of 8000 features have lower precision and recall compared to our models. For majority classes, our model poses better performance such as class 0 1 3 and 4. However, in the minority class, our model has better results on recall but slightly lower in precision. It's due to the fact that we have different approaches in both pre-processing data and building models. For light gbm, this algorithm although yields negligibly better result in precision and recall; however the time complexity of our model is far more better compared with both algorithms.